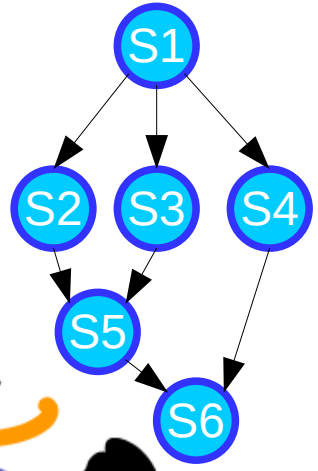
Clase 10 – Concurrencia

Programación concurrente.
Aplicaciones
Creación de un Threads
Ciclo de vida un Thread
Extendiendo de la clase Thread
Implementando la interfaz Runnable
Compartiendo objetos entre distintos Threads
Sincronización de Threads
Métodos synchronized
Métodos estáticos sincronizados
Comunicación entre Threads: wait, notifyAll, notify
La sentencia syncronized.
Bloqueo selectivo
Semáforos
Métodos acquire y release

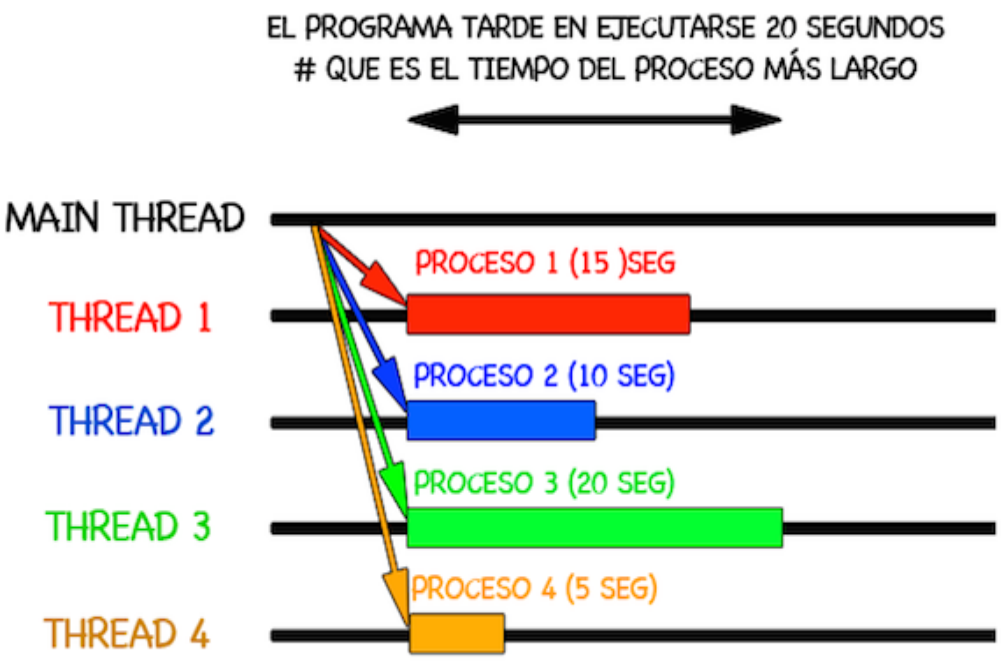


Concurrencia
~2022~





Clase 10 - Concurrencia



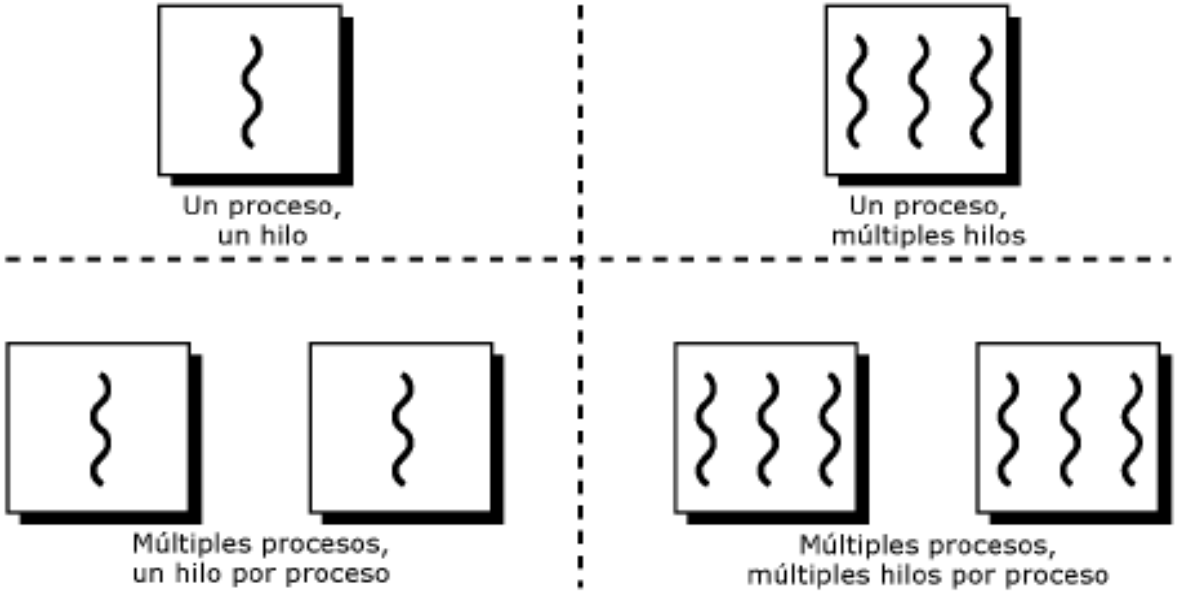
Una gran cantidad de problemas de programación pueden resolverse utilizando programación secuencial. Sin embargo, para algunos problemas, resulta conveniente e incluso esencial ejecutar varias partes del programa en paralelo, de modo que dichas partes parezcan estarse ejecutando concurrentemente o, si hay disponibles varios procesadores, se ejecuten realmente de manera simultánea.

Programación Concurrente

Se conoce por programación concurrente a la rama de la informática que trata de las técnicas de programación que se usan para expresar el paralelismo entre tareas y para resolver los problemas de comunicación y sincronización entre procesos.

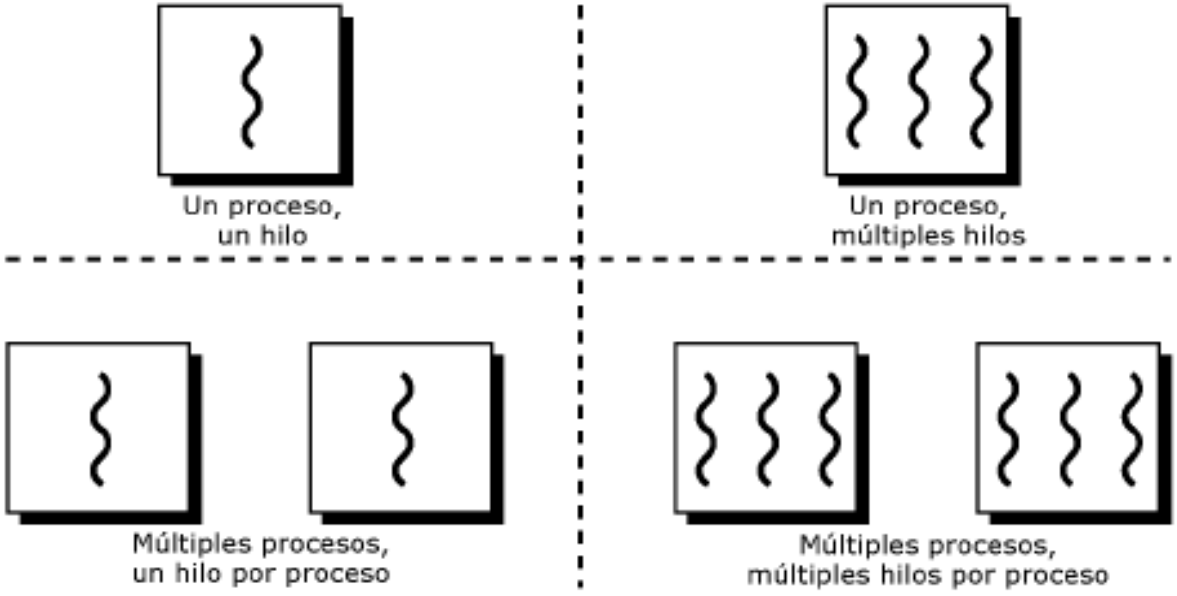
El principal problema de la programación concurrente corresponde a no saber en que orden se ejecutan los programas (en especial los programas que se comunican). Se debe tener especial cuidado en que este orden no afecte el resultado de los programas.

Programación Concurrente



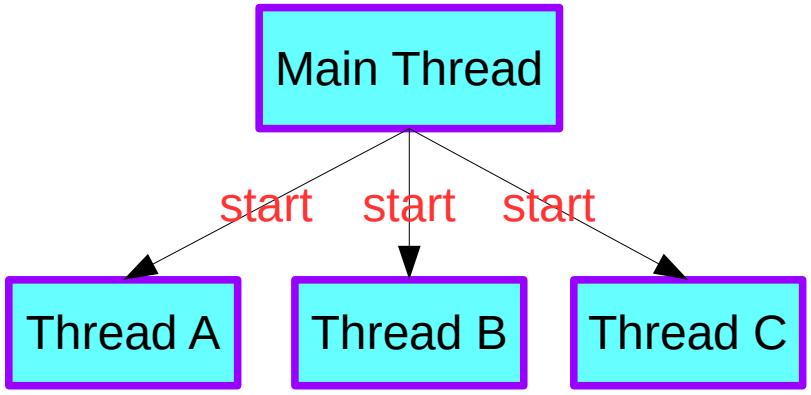
Una forma muy sencilla de implementar la concurrencia es en el nivel del sistema operativo, utilizando *procesos*. Un proceso es un programa autocontenido que se ejecuta en su propio espacio de direcciones. Un S.O. *multitarea* puede ejecutar más de un proceso (programa) simultáneamente, conmutando periódicamente la CPU de un proceso a otro, lo que aparenta que cada proceso se ejecuta por separado y al mismo tiempo.

Programación Concurrente



Los sistemas concurrentes, como los que utiliza Java, comparten recursos tales como la memoria, E/S, por lo que la dificultad está en coordinar el uso de estos recursos entre las distintas tareas dirigidas por hilos, de modo que no haya más de una tarea en cada momento que pueda acceder a un determinado recurso.

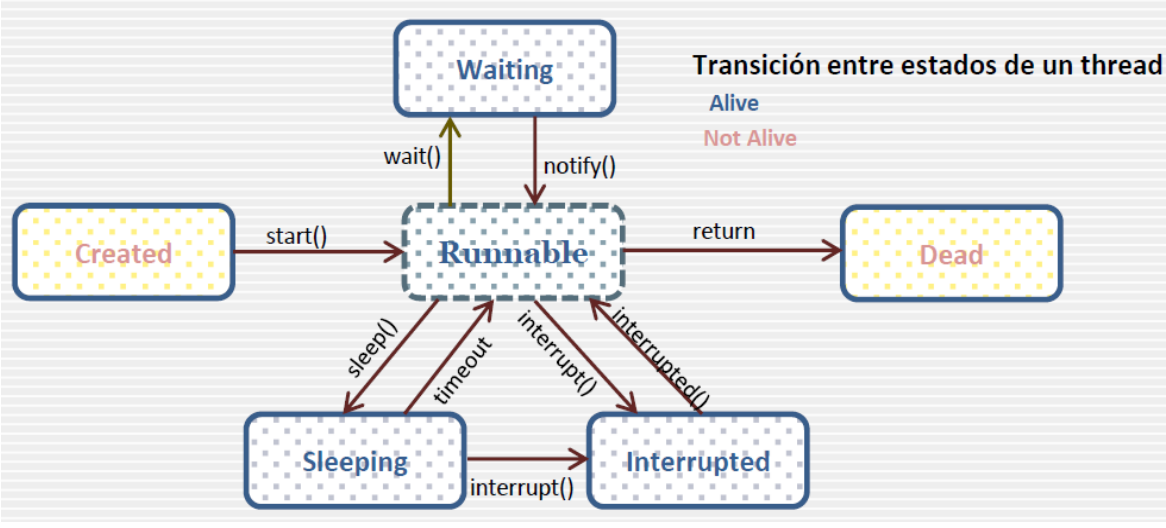
Hilos (Threads) en Java



Java proporciona un API para el uso de hilos: la **clase Thread,** dentro del paquete java.lang.Thread.

Cuando arranca un programa existe un hilo principal (main), y luego se pueden generar nuevos hilos.

La clase Thread dispone de una serie de métodos para caracterizar el thread/hilo en el programa: isAlive(), isDaemon(), setName(), setDaemon(), run(), etc.



Creación de un Thread

Para crear un Thread (hilo de control), se crea un objeto de tipo Thread:

```
Thread trabajador = new Thread();
```

Tras crearlo, se puede configurar y ejecutar.

Para ejecutarlo se invoca el método **`start()`**.

El método **`start()`** lanza un nuevo hilo de control basado en los datos del objeto Thread, y luego retorna.

```
trabajador.start();
```

Al lanzarse el método **`start()`**, la máquina virtual invoca al nuevo método **`run()`** del hilo de control, con lo que el hilo se activa.

Creación de un Thread

```
public class HiloBasico extends Thread
{
    int cont;

    HiloBasico(int c)
    {
        cont = c;
    }

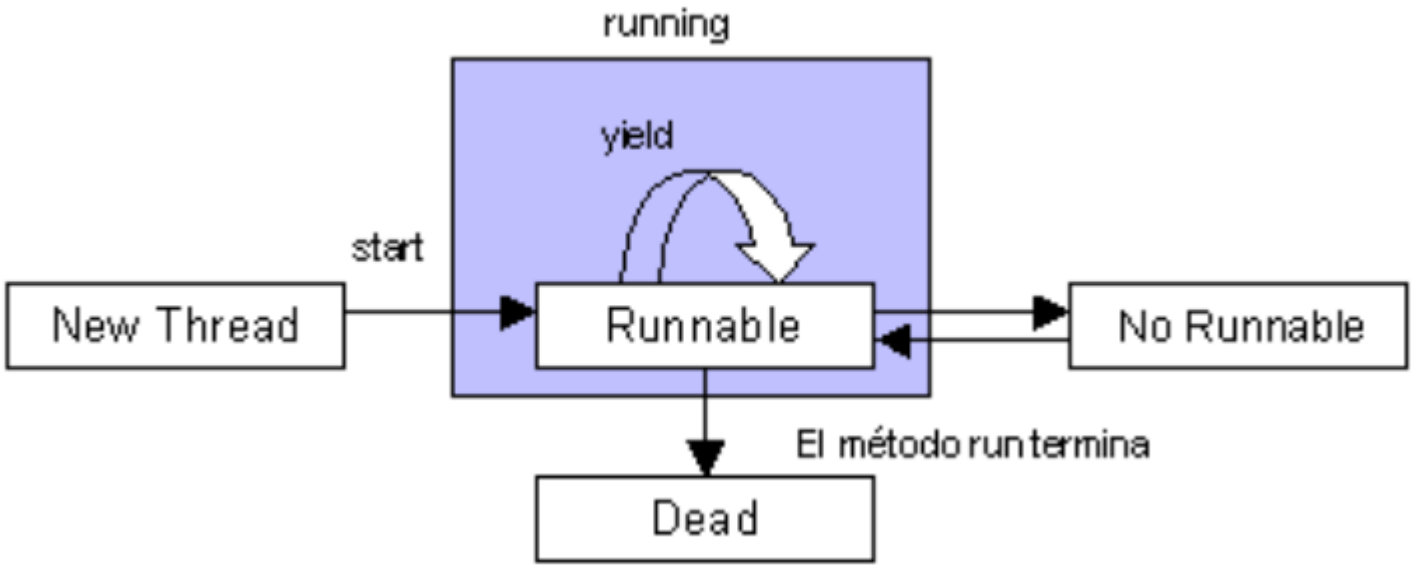
    public void run()
    {
        while (true)
        {
            System.out.println(cont);
        }
    }

    public static void main(String[] args)
    {
        new HiloBasico(0).start();
        new HiloBasico(1).start();
    }
}
```

Sólo se puede invocar **`start()`** una vez para cada hilo (de invocarlo nuevamente se lanza la excepción IllegalThreadStateException.

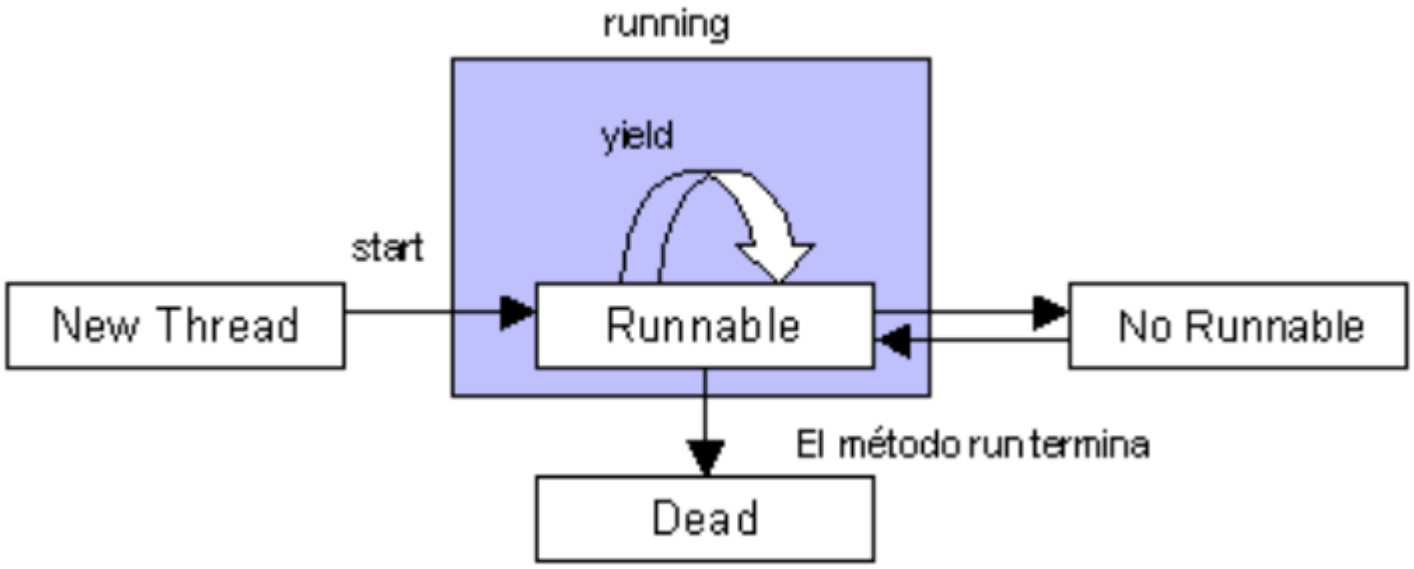
Cuando se retorna de un método **`run()`** de un hilo, el hilo ha finalizado.

Ciclo de Vida de un Thread



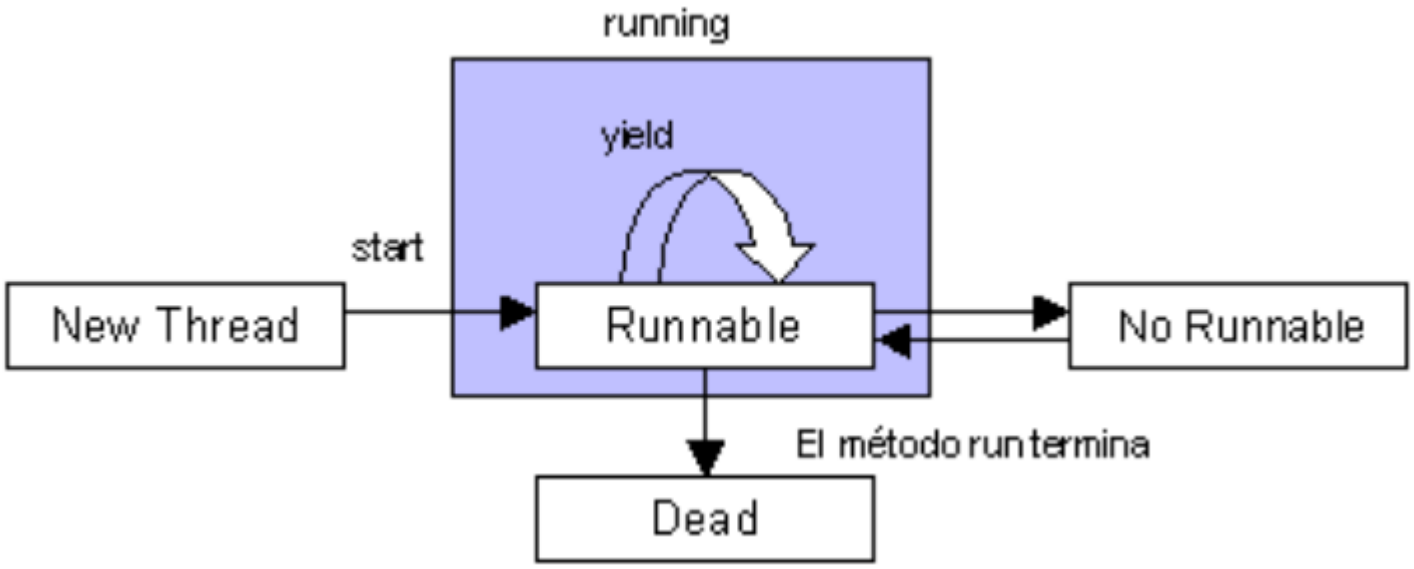
Cuando se instancia la clase Thread (o una subclase) se crea un nuevo Thread que está en su estado inicial ('New Thread' en el gráfico). En este estado es simplemente un objeto más. No existe todavía el thread en ejecución.

Ciclo de Vida de un Thread



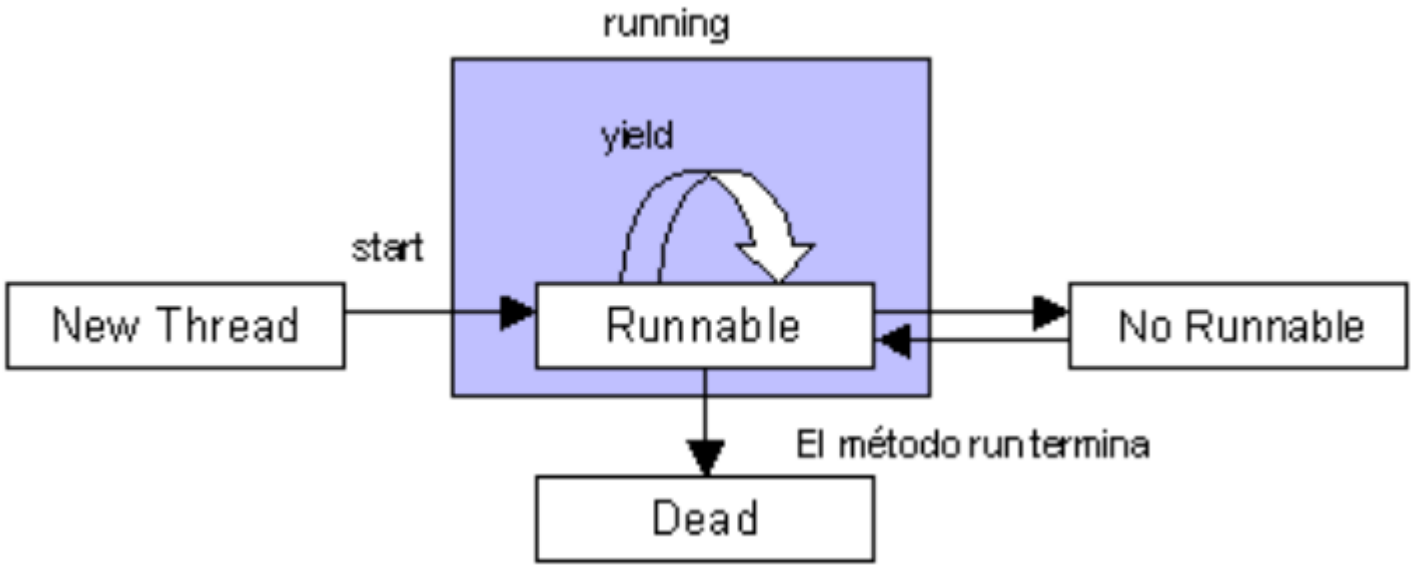
El único método que puede invocarse sobre él es el método start(). Cuando se invoca el método start() sobre el hilo, el sistema crea los recursos necesarios, lo planifica (le asigna prioridad) y llama al método run(). En este momento el hilo está corriendo, se encuentra en el estado ‘runnable’.

Ciclo de Vida de un Thread



Si el método run() invoca internamente el método sleep() o wait() o el hilo tiene que esperar por una operación de entrada/salida, entonces el hilo pasa al estado 'no runnable' (no ejecutable) hasta que la condición de espera finalice. Durante este tiempo el sistema puede ceder control a otros hilos activos.

Ciclo de Vida de un Thread



Por último cuando el método run finaliza el hilo termina y pasa a la situación 'Dead' (Muerto).

Ciclo de Vida de un Thread

La finalización de un hilo suele ocurrir cuando se termina de ejecutar el método run() 👍

No obstante, hay otras tres razones por las que un hilo puede terminar:

- En algún momento de la ejecución del método run() se ha generado una excepción que nadie ha capturado. La excepción se propaga hasta el propio método run(). Si tampoco éste tiene un manejador para la excepción, el método run() finaliza abruptamente, terminando la ejecución del hilo. 👎
- Si se llama al método stop() del hilo. 👎
- Cuando se llama al método destroy() del hilo. 👎

En cualquiera de los casos, el hilo finaliza su ejecución, y deja de ser tenida en cuenta por el planificador.

Creación de un Thread

La implementación estándar de **`Thread.run()`**, no hace nada.

Para obtener un hilo que haga algo, se debe hacer una de estas dos posibilidades:

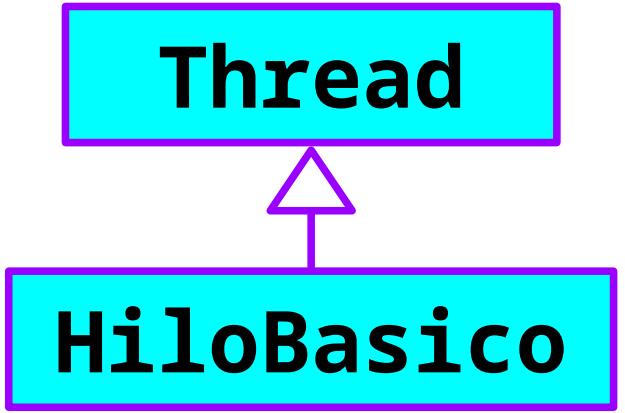
1) Crear una clase que extienda de Thread y sobreescribir el método run.

2) Crear una clase que implemente la interfaz Runnable y crear un objeto de tipo Thread a partir de éste.




```
Thread unThread = new Thread ( new Mi_Runnable());
```

Extendiendo de la clase **Thread**



1) Se crea una clase que extienda de Thread
2) Se implementa el método run()
3) Luego de instanciar la clase creada, se invoca el método start(), que a su vez invoca el método run()

```
public class HiloBasico extends Thread
{
    int cont;

    HiloBasico(int c)
    {
        cont = c;
    }

    public void run()
    {
        while (true)
        {
            System.out.println(cont);
        }
    }

    public static void main(String[] args)
    {
        new HiloBasico(0).start();
        new HiloBasico(1).start();
    }
}
```

Extendiendo de la clase **Thread**

La ejecución de la aplicación causa (en teoría) la salida por pantalla de líneas con 0's y 1's entrelazados. Eso se debe a que la máquina virtual distribuye el tiempo del procesador entre los dos Hilos, de modo que según quién tenga el procesador se escribirá un 0 o un 1.

Este modo de implementar nuevos hilos tiene una desventaja, debido a la herencia simple.

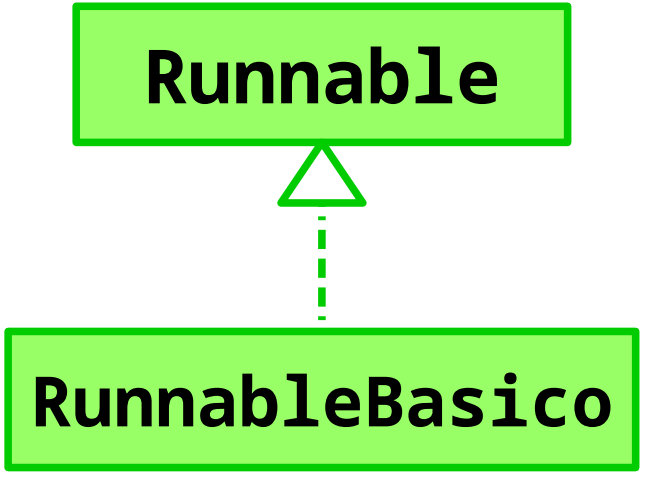
```
public class HiloBasico extends Thread
{
    int cont;

    HiloBasico(int c)
    {
        cont = c;
    }

    public void run()
    {
        while (true)
        {
            System.out.println(cont);
        }
    }

    public static void main(String[] args)
    {
        new HiloBasico(0).start();
        new HiloBasico(1).start();
    }
}
```

Implementando **Runnable**




```java
public class RunnableBasico implements Runnable
{
    int cont;

    RunnableBasico(int c)
    {
        cont = c;
    }

    public void run()
    {
        while (true)
        {
            System.out.println(cont);
        }
    }

    public static void main(String[] args)
    {
        new Thread(new RunnableBasico(0)).start();
        new Thread(new RunnableBasico(1)).start();
    }
}
```

```
Thread unThread = new Thread ( new RunnableBasico());
```

Implementando **Runnable**

Es posible comenzar la ejecución del método run() de cualquier objeto que implemente el interfaz en un hilo independiente. Para eso, es suficiente crear un nuevo objeto de la clase Thread, pasándole en el constructor el objeto que se desea ejecutar.

Cuando se llame al método start() de esa hebra, el método que se ejecutará será, realmente, el método run() del objeto pasado en el constructor.

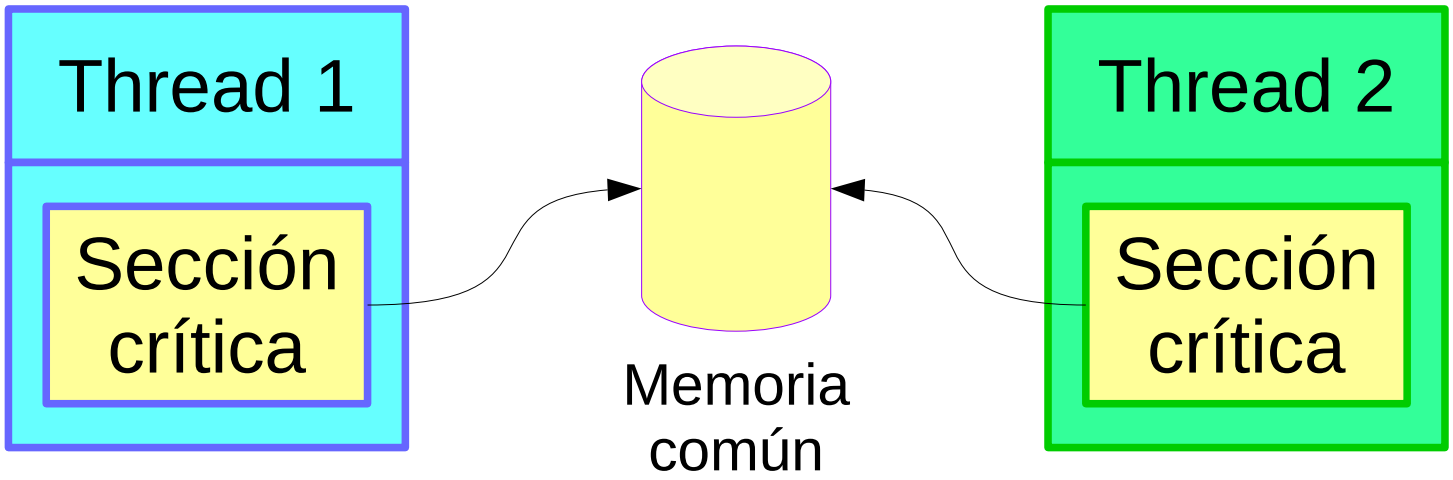
```
public class RunnableBasico implements Runnable
{
    int cont;

    RunnableBasico(int c)
    {
        cont = c;
    }

    public void run()
    {
        while (true)
        {
            System.out.println(cont);
        }
    }

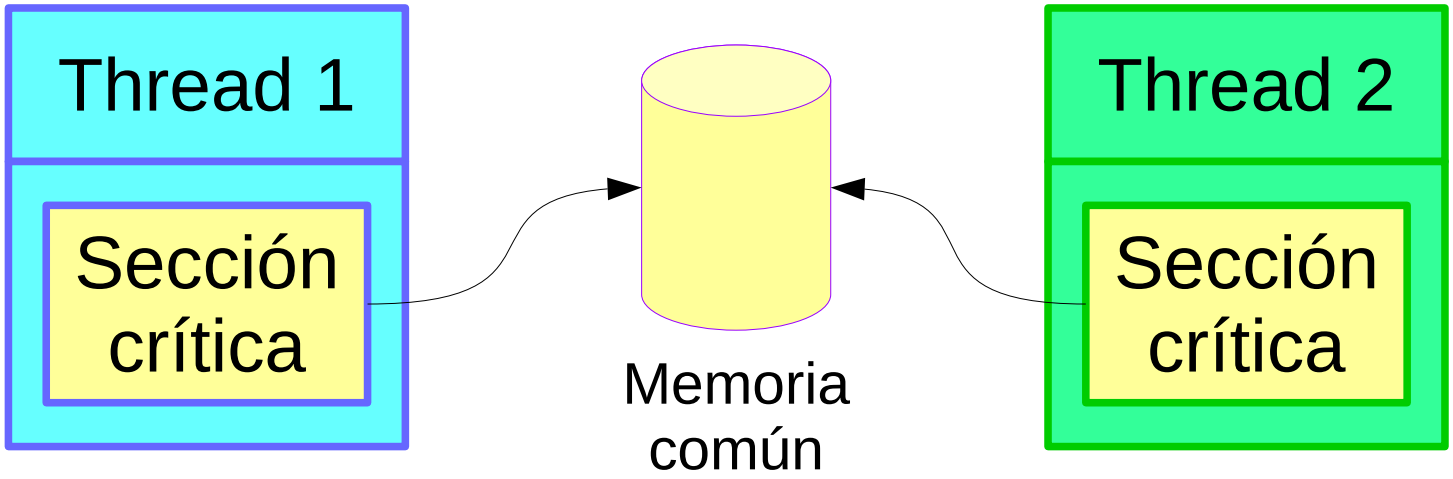
    public static void main(String[] args)
    {
        new Thread(new RunnableBasico(0)).start();
        new Thread(new RunnableBasico(1)).start();
    }
}
```

Compartiendo objetos entre distintos Threads



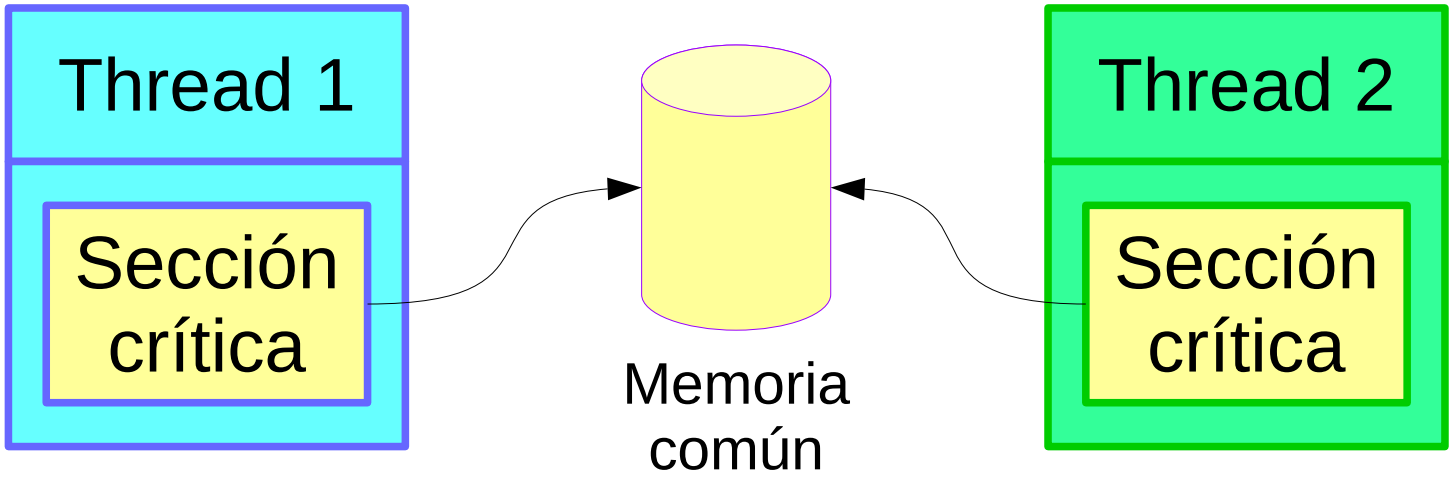
Cuando varios Threads *comparten un mismo objeto y modifican su estado*, pueden ocurrir inconsistencias en la visión que tenga cada thread del objeto.

Compartiendo objetos entre distintos Threads



Al estar en un ambiente concurrente, si no se establece una ***sincronización*** en el acceso al objeto compartido (de a un thread por vez), los resultados son imprevisibles.

Compartiendo objetos entre distintos Threads



Se llama ***sección crítica*** a los segmentos de código dentro de un programa que acceden a zonas de memoria comunes desde distintos threads que se ejecutan concurrentemente.

Ejemplo

```java
public class Contador
{
    private long valor = 0;

    public void incrementa()
    {
        long aux;
        aux = valor;
        aux ++;
        valor = aux;
    }

    public long getValor()
    {
        return valor;
    }
}
```

Sección crítica

```java
public class Contable extends Thread
{
    Contador contador;

    public Contable(Contador c)
    {
        this.contador = c;
    }

    public void run()
    {
        int i;
        for(i = 1; i <= 100000; i++)
        {
            contador.incrementa();
        }
        System.out.println("Contador hasta ahora: "
                           + contador.getValor());
    }
}
```

Ejemplo

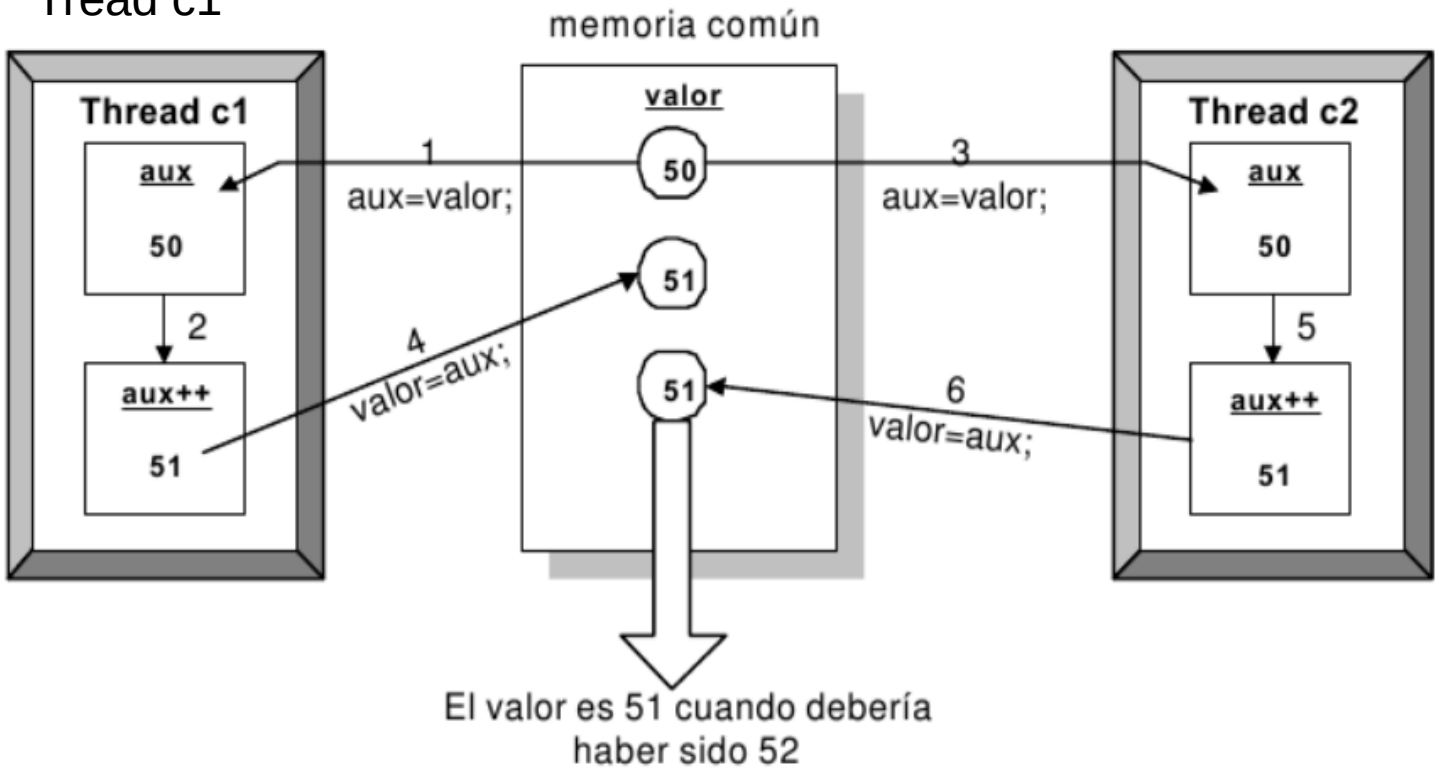
```
class ThreadSinSync
{
   public static void main(String arg[])
   {
      Contable c1, c2;
      Contador c = new Contador();
      c1 = new Contable(c);
      c2 = new Contable(c);
      c1.start();
      c2.start();
   }
}
```

La salida por pantalla es:
Contado hasta ahora: 124739
Contado hasta ahora: 158049
cuando debería haber sido: en la primera línea un número comprendido entre 100000 y 200000; y en la segunda línea el valor 200000.

Por qué sucedió eso?

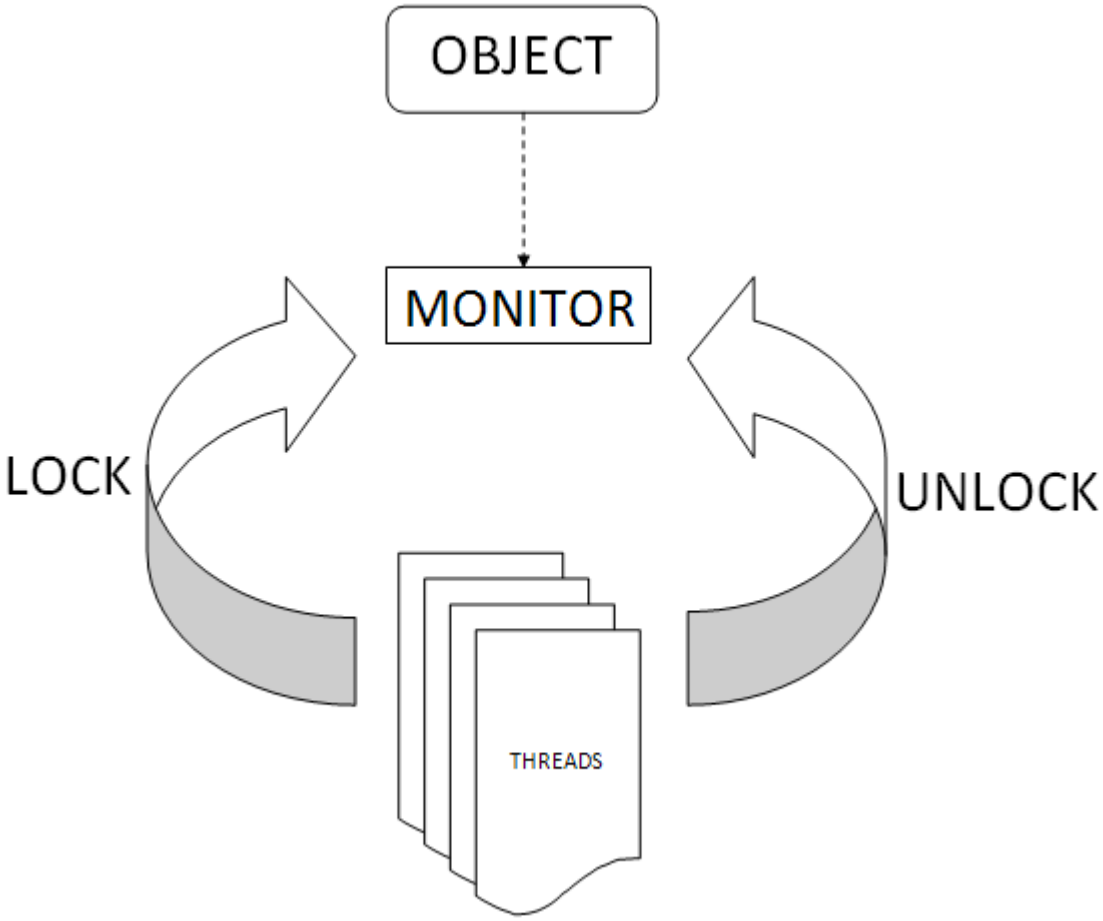
```
class Contador
{
    private long valor = 0;
    public void incrementa()
    {
        long aux;
        aux = valor;
        aux++;
        valor = aux;
    }
    public long getValor()
    {
        return valor;
    }
}
```

El problema consiste en no llegar a terminar la ejecución del método incrementa() cuando esta siendo invocado por el Tread c1



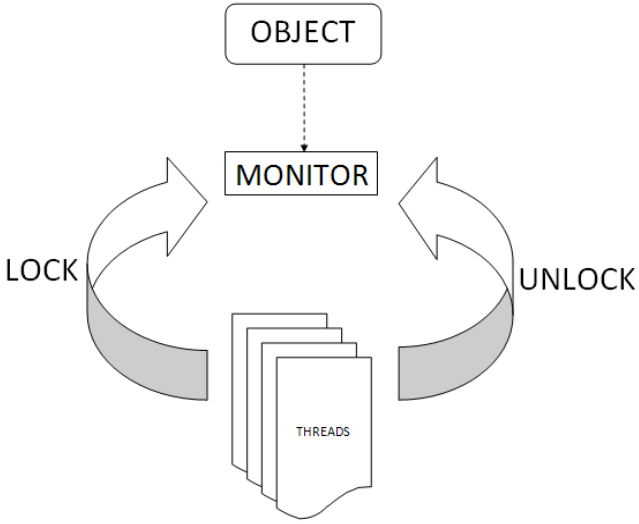
	valor	aux
1	50	50
2	50	51
3	50	50
4	50	50
5	50	51
6	51	51

Monitores



En la programación paralela, **los monitores** son objetos destinados a ser usados sin peligro por más de un hilo de ejecución. La característica que principalmente los define es que sus **métodos son ejecutados con exclusión mutua.** Lo que significa, que en cada momento en el tiempo, un hilo como máximo puede estar ejecutando cualquiera de sus métodos.

Monitores - Componentes



Un monitor tiene cuatro componentes: inicialización, métodos privados, métodos del monitor y cola de entrada.

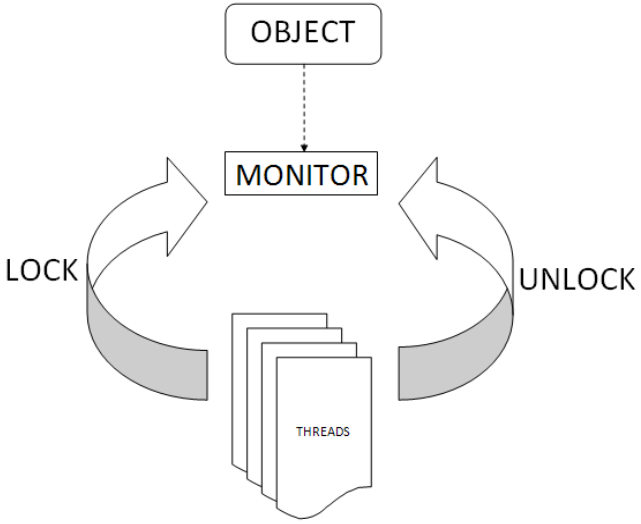
Inicialización: contiene el código a ser ejecutado cuando el monitor es creado

Métodos privados: contiene los métodos privados, que sólo pueden ser usados desde dentro del monitor y no son visibles desde fuera

Métodos del monitor: son los procedimientos que pueden ser llamados desde fuera del monitor.

Cola de entrada: contiene a los hilos que han llamado a algún método del monitor pero no han podido adquirir permiso para ejecutarlos aún.

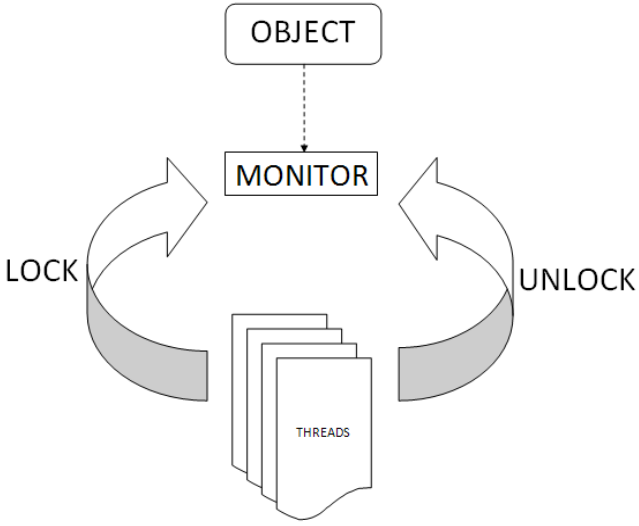
Monitores – Exclusión mutua



Los monitores están pensados para ser usados en **entornos multiproceso** o multihilo, y por lo tanto muchos procesos o threads pueden llamar a la vez a un método del monitor. Los monitores garantizan que en cualquier momento, **a lo sumo un thread** puede estar ejecutando dentro de un monitor.

Ejecutar dentro de un monitor significa que sólo un thread estará en estado de ejecución mientras dura la llamada a un método del monitor. Para **garantizar la integridad de los datos privados**, el monitor hace cumplir la exclusión mutua implícitamente, de modo que sólo un método esté siendo ejecutado a la vez. De esta forma, si **un thread** llama a un método mientras otro thread está dentro del monitor, **se bloqueará y esperará** en la cola de entrada hasta que el monitor quede nuevamente libre.

Monitores – Exclusión mutua



Para que resulten útiles en un entorno de concurrencia, los monitores deben incluir algún tipo de forma de **sincronización**. Por ejemplo, supóngase un thread que está dentro del monitor y necesita que se cumpla una condición para poder continuar la ejecución. En ese caso, se debe contar con un mecanismo de **bloqueo del thread**, a la vez que se debe **liberar el monitor para ser usado por otro hilo**. Más tarde, cuando la condición permita al thread bloqueado continuar ejecutando, debe poder ingresar en el monitor en el mismo lugar donde fue suspendido. Para esto los monitores poseen variables de condición que son accesibles sólo desde adentro.

Estructura sintáctica de un Monitor

```
class Mi_Monitor
{
    //definir aquí datos protegidos por el monitor

    public Mi_Monitor()
    {…} //constructor

    public synchronized tipo1 metodo1() throws InterruptedException
    {
        while(!condicion1)
            wait();
        …
        notifyAll();
    }

    public synchronized tipo2 metodo2() throws InterruptedException
    {...}
```

Estructura sintáctica de un Monitor

Analicemos cada componente por parte y en forma progresiva

Sincronización de Threads



Para evitar corromper la zona crítica de un recurso compartido (objeto referido por varios threads en forma concurrente) es necesario que los mensajes enviados al recurso por parte de cada thread se realicen de a uno por vez.

A esta acción sincronizada se la denomina ***bloqueo de un objeto.***

Para indicar esta acción se utiliza la sentencia

`synchronized`

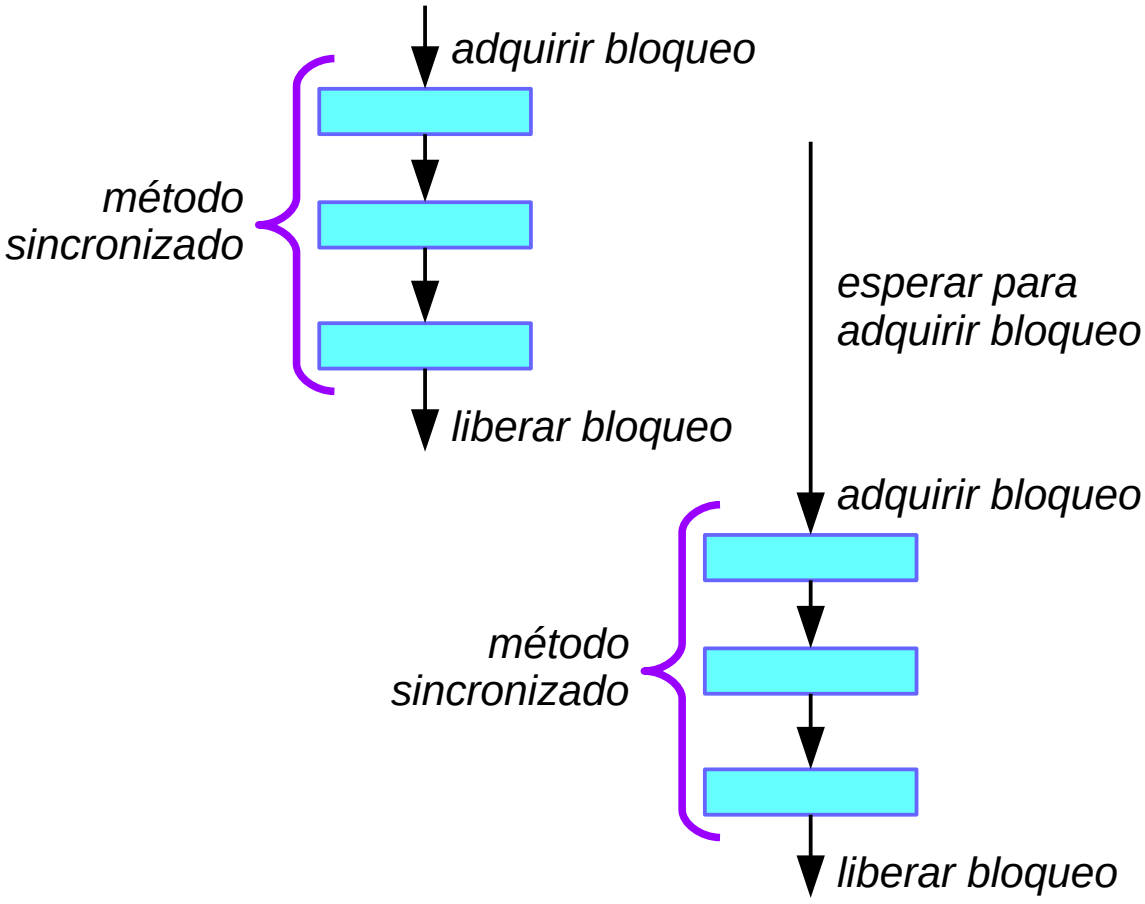
Sincronización de Threads



Cada thread debe ***adquirir el bloqueo de un objeto*** antes de enviarle un mensaje que pueda corromper la zona crítica.

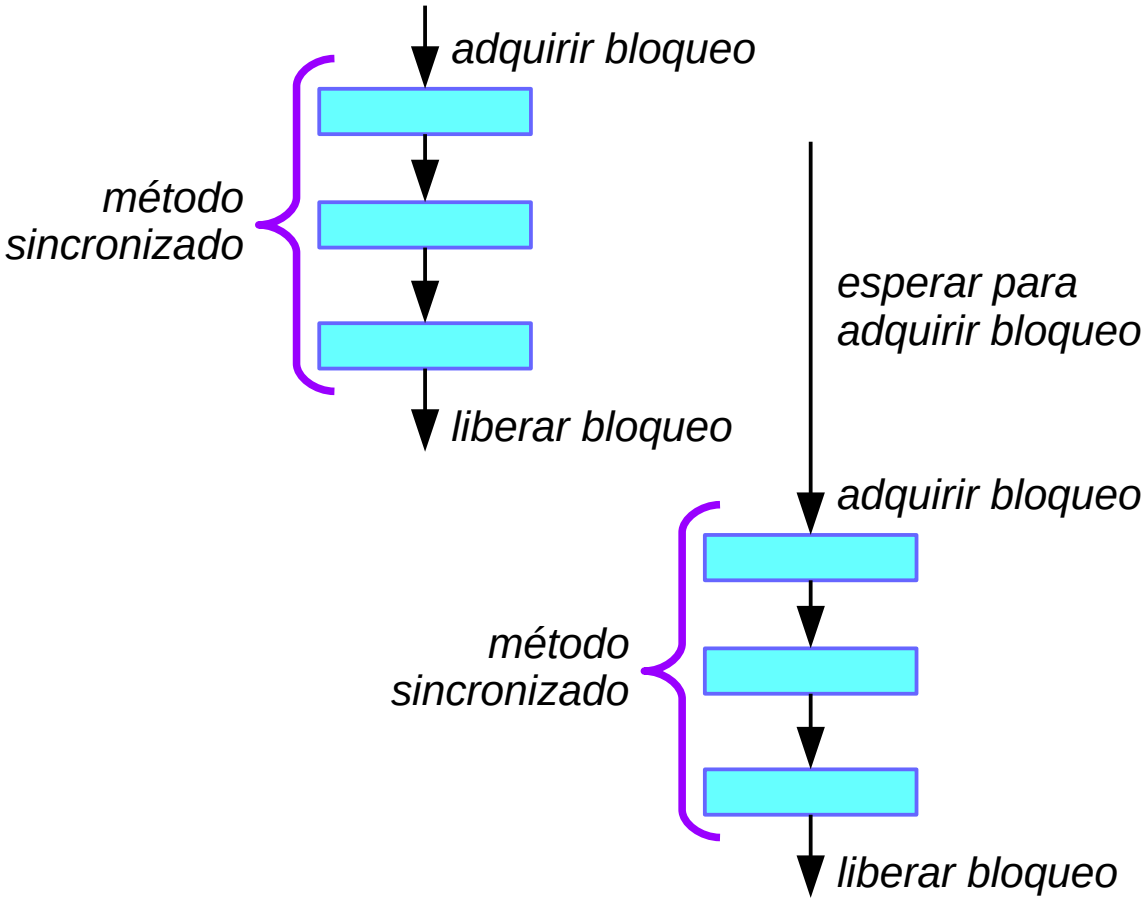
Adquirir el bloqueo de un objeto impide que cualquier otro thread lo adquiera, hasta que el propietario lo libere.

Métodos **synchronized**



Si un hilo invoca a un método synchronized sobre un objeto (recurso compartido), en primer lugar se adquiere el bloqueo de ese objeto, se ejecuta el cuerpo del método y después se libera el bloqueo.

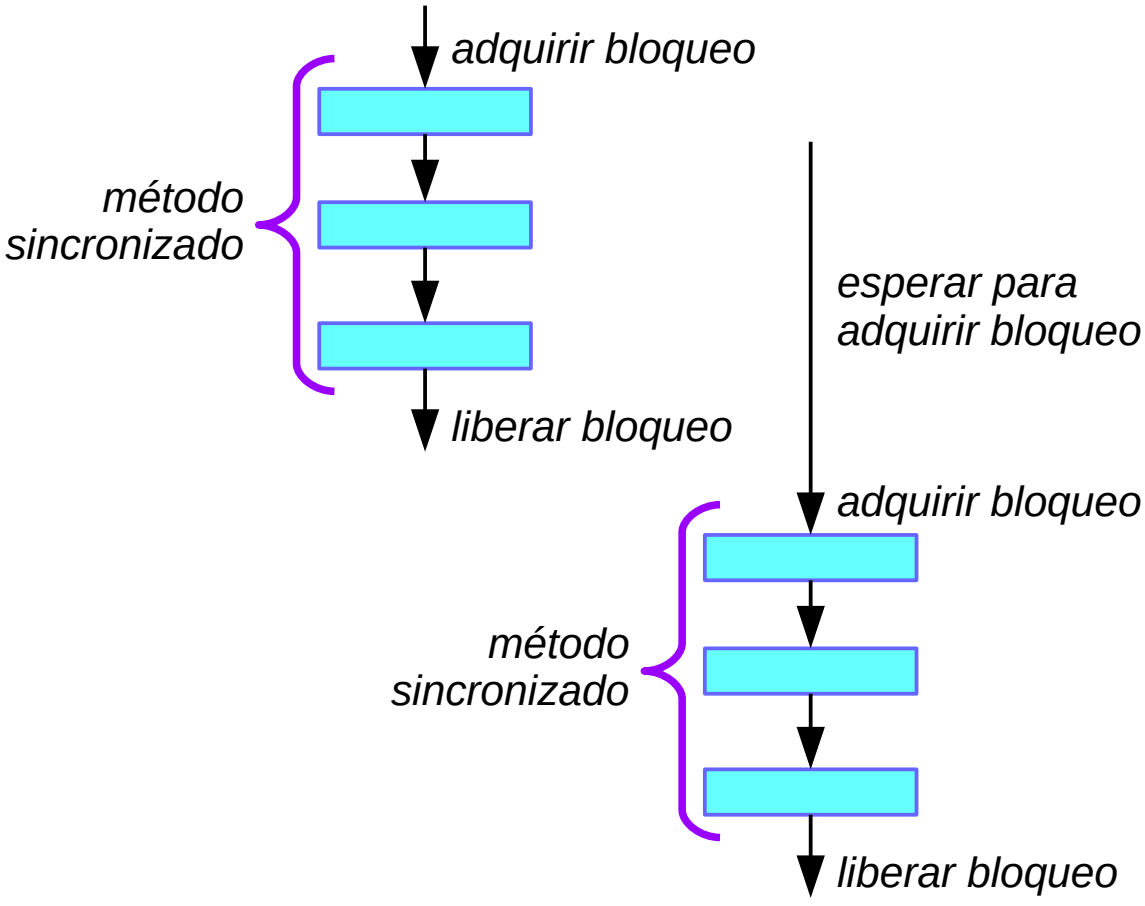
Métodos **synchronized**



Otro hilo que invoque un método synchronized sobre ese mismo objeto se bloqueará hasta que el hilo anterior libere el bloqueo. O sea, debe esperar hasta que el recurso esté disponible.

Si varios hilos esperan por un recurso, **no se sabe cuál** será el siguiente en obtenerlo.

Métodos **synchronized**



Ejecución mutuamente exclusiva en el tiempo.

Los accesos NO sincronizados no esperan por ningún bloqueo.

La posesión de los bloqueos es por hilo. Al invocar a un método sincronizado desde el interior de otro método sincronizado sobre el mismo objeto, se procederá sin bloqueo. Se libera el bloqueo cuando el método sincronizado más externo retorne.

Métodos **synchronized**

Sintaxis

```
public synchronized tipo metodo()
{
   . . .
}
```

Métodos **`synchronize`**: implicancias

1)Los constructores no necesitan ser sincronizados (de hecho no pueden declararse sincronizados)

2)El acceso a los campos debe hacerse a través métodos públicos exclusivamente, ya que es la única manera de evitar corromper la zona crítica

3)No hay garantías del orden en que se ejecutarán los métodos sincronizados al competir por el recurso

4)Cuando una clase extendida redefine a un método syncronizado (en la clase base), el nuevo método puede ser sincronizado o no.

- Los requerimientos de sincronización son parte de la implementación de una clase.

Métodos estáticos sincronizados

Los métodos estáticos también se pueden declarar sincronizados.

Dos hilos no pueden ejecutar métodos estáticos sincronizados de la misma clase al mismo tiempo.

La adquisición del bloqueo en un método estático sincronizado no afecta a los objetos de esa clase. Solo se bloquean otros métodos estáticos sincronizados.

Métodos **synchronized**: Ejemplo *productor - consumidor*

```
public class Contenedor
{
    private int valor;

    public Contenedor()
    {
        super();
    }

    public synchronized void put(int nv)
    {
        valor = nv;
        System.out.println("metido el valor: " + valor);
    }

    public synchronized int get()
    {
        System.out.println("sacado el valor: " + valor);
        //valor = 0;
        return valor;
    }
}
```

Métodos **synchronized**: Ejemplo *productor - consumidor*

```java
public class Productor extends Thread
{
    private Contenedor contenedor;

    public Productor(Contenedor c)
    {
        contenedor = c;
    }

    public void run()
    {
        int i;
        for (i = 0; i < 10; i++)
        {
            contenedor.put(i);
            try
            {
                sleep((int) (Math.random() * 100));
            }
            catch (InterruptedException e)
            {
            }
        }
    }
}
```

```java
public class Consumidor extends Thread
{
    private Contenedor contenedor;

    public Consumidor(Contenedor c)
    {
        contenedor = c;

    }

    public void run()
    {
        int value = 0;
        int i;
        for (i = 0; i < 10; i++)
        {
            value = contenedor.get();
            try
            {
                sleep((int) (Math.random() * 100));
            }
            catch (InterruptedException e)
            {
            }
        }
    }
}
```

Métodos **synchronized**: Ejemplo *productor - consumidor*

```
public class Prueba
{
    public Prueba()
    {
        super();
    }

    public static void main(String args[])
    {
        Contenedor recursoCompartido = new Contenedor();
        Productor productor = new Productor(recursoCompartido);
        Consumidor consumidor = new Consumidor(recursoCompartido);
        productor.start();
        consumidor.start();
    }
}
```

Métodos **synchronized**: Ejemplo *productor - consumidor*

Tanto el hilo productor como el consumidor comparten el mismo objeto de la clase Caja. Para asegurar la exclusión mutua en la zona crítica (la que accede a valor), se declaran los métodos get() y put() como synchronized.

Pero esto no es suficiente para que el programa funcione correctamente.

la salida por pantalla podría ser la siguiente:

```
metido el valor: 0
sacado el valor: 0
sacado el valor: 0
metido el valor: 1
sacado el valor: 1
sacado el valor: 1
metido el valor: 2
metido el valor: 3
sacado el valor: 3
metido el valor: 4
sacado el valor: 4
metido el valor: 5
sacado el valor: 5
sacado el valor: 5
metido el valor: 6
metido el valor: 7
metido el valor: 8
sacado el valor: 8
sacado el valor: 8
metido el valor: 9
```

Métodos **`synchronized`**: Ejemplo *productor - consumidor*

De alguna forma habría que asegurar que no ocurran errores, tales como sacar un elemento de la caja cuando está vacía.

En nuestro ejemplo, el error se refleja en los renglones destacados

la salida por pantalla podría ser la siguiente:

```
metido el valor: 0
sacado el valor: 0
sacado el valor: 0
metido el valor: 1
sacado el valor: 1
sacado el valor: 1
metido el valor: 2
metido el valor: 3
sacado el valor: 3
metido el valor: 4
sacado el valor: 4
metido el valor: 5
sacado el valor: 5
sacado el valor: 5
metido el valor: 6
metido el valor: 7
metido el valor: 8
sacado el valor: 8
sacado el valor: 8
metido el valor: 9
```

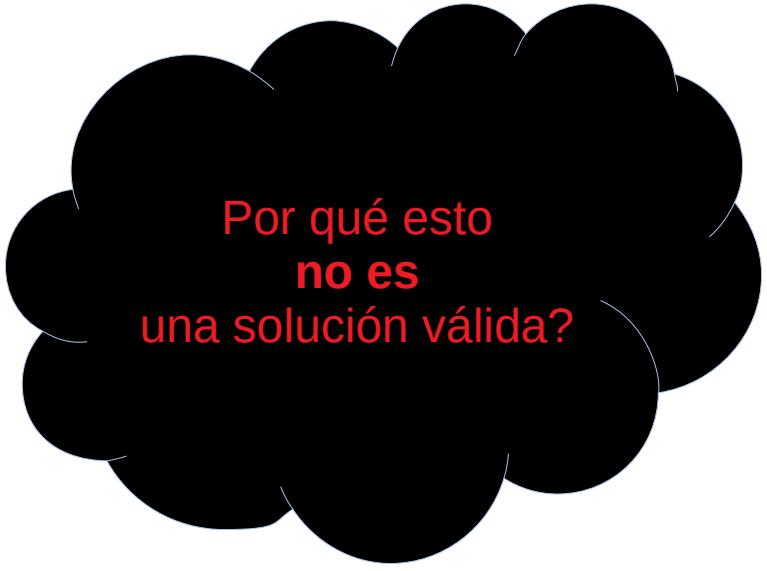
Solución **aparente** al problema anterior

```
public class Caja
{
    private int valor;
    private boolean disponible = false;

    public Caja()
    {
        super();
    }

    public synchronized void meter(int nv)
    {
        System.out.println("intentando meter el valor: " + nv);
        if (!disponible)
        {
            valor = nv;
            disponible = true;
            System.out.println("---> metido el valor: " + valor);
        }
    }

    public synchronized void sacar()
    {
        System.out.println("intentando sacar el valor disponible ");
        if (disponible)
        {
            System.out.println("---> sacado el valor: " + valor);
            valor = 0;
            disponible = false;
        }
    }
}
```



Solución aparente al problema anterior

```
class Prod2 extends Thread
{
    Caja c;

    public Prod2(Caja nc)
    {
        c = nc;
    }

    public void run()
    {
        int i;
        for (i = 1; i <= 10; i++)
        {
            c.meter(i);
        }
    }
}
```

```
class Cons2 extends Thread
{
    Caja c;

    public Cons2(Caja nc)
    {
        c = nc;
    }

    public void run()
    {
        int i;
        for (i = 1; i <= 10; i++)
        {
            c.sacar();
        }
    }
}
```

Solución aparente al problema anterior

```
class ProdCons
{
    public static void main(String argum[])
    {
        Caja cj = new Caja();
        Prod2 p = new Prod2(cj);
        Cons2 c = new Cons2(cj);
        p.start();
        c.start();
    }
}
```




```
intentando meter el valor: 1
---> metido el valor: 1
intentando meter el valor: 2
intentando meter el valor: 3
intentando meter el valor: 4
intentando meter el valor: 5
intentando meter el valor: 6
intentando meter el valor: 7
intentando meter el valor: 8
intentando meter el valor: 9
intentando meter el valor: 10
intentando sacar el valor disponible
---> sacado el valor: 1
intentando sacar el valor disponible
intentando sacar el valor disponible
intentando sacar el valor disponible
intentando sacar el valor disponible
intentando sacar el valor disponible
intentando sacar el valor disponible
intentando sacar el valor disponible
intentando sacar el valor disponible
intentando sacar el valor disponible
Process exited with exit code 0.
```

Después de sacar el valor 1, el consumidor ha seguido su ejecución en el bucle y como no se mete ningún valor por el consumidor, termina su ejecución, el productor introduce el siguiente valor (el 2), y sigue su ejecución hasta terminar el bucle; como el consumidor ya ha terminado su ejecución no se pueden introducir más valores y el programa termina.

Comunicación entre Threads: **`wait, notifyAll, notify`**

evitar interferencias

El mecanismo de bloqueo de synchornized es suficiente para evitar que los hilos interfieran entre sí, pero también necesitamos formas de comunicación entre hilos.

espera de un thread

En el caso del ejemplo, lo que hace falta es un mecanismo que produzca la espera del productor si la caja tiene algún valor (disponible) y otro que frene al consumidor si la caja está vacía (no disponible):

Comunicación entre Threads: **wait, notifyAll, notify**

Para este propósito el método wait deja un thread esperando hasta que alguna condición ocurra, y los métodos de notificación notifyAll y notify avisan a los threads que estan esperando que algo ha ocurrido que podría satisfacer la condición. Los métodos wait y los de notificación están definidos en Object.

```
synchronized void doWhenCondition()
{
  while (!condition)
    wait();
  //hace algo cuando la condición es verdadera
}
```

Análisis del código: el hilo se pone a dormir

```
synchronized void doWhenCondition()
{
  while (!condition)
    wait();
  //hace algo cuando la condición es verdadera
}
```

1.- Todo se ejecuta dentro de un código sincronizado

Si no fuera así, el estado del objeto no sería estable. Por ejemplo, si el método no fuera declarado synchronized, después de la sentencia while, no habría garantía que la condición se mantenga verdadera, otro thread podría cambiar la situación que la condición testea.

Análisis del código: el hilo se pone a dormir

```
synchronized void doWhenCondition()
{
  while (!condition)
    wait();
  //hace algo cuando la condición es verdadera
}
```

2.- Suspende el thread y libera el bloqueo del recurso compartido

Uno de los aspecto más importantes de la definición de wait es que cuando suspende al thread, libera atómicamente el bloqueo sobre el objeto. Suceden juntas, de forma indivisible. Si no, habría riesgo de competencia: podría suceder una notificación después de que se liberara el bloqueo pero antes de que se suspendiera el hilo. La notificación no habría afectado el hilo, perdiéndose. Cuando el hilo se reinicia después de la notificación, el bloqueo se vuelve a adquirir atómicamente.

Análisis del código: el hilo se pone a dormir

```
synchronized void doWhenCondition()
{
  while (!condition)
    wait();
  //hace algo cuando la condición es verdadera
}
```

3.- La condición de prueba debe estar siempre en un ciclo de repetición

La condición de prueba debería estar siempre en un bucle. Nunca debe asumirse que ser despertado significa que la condición se ha cumplido, ya que puede haber cambiado de nuevo desde que se cumplió. No debe cambiar un while por un if.

Análisis del código: se notifica a los hilos que despierten

```
synchronized void
doWhenCondition()
{
    while (!condition)
        wait();
    //hace algo cuando condición es verdadera
}
```

```
    // se termina de usar
    // el recurso compartido
    notifyall();

    …
}
```

4.- Conviene usar notifyAll() en vez de notify()

Al utilizar notifiAll se despiertan todos los hilos en espera (independientemente de su condición), y notify selecciona sólo un hilo para despertar (uno cualquiera). Si se despertara uno que no satisface la condición modificada, volvería a dormir, y el que hubiera debido despertarse nunca lo hizo.
El uso de notify es una optimización que sólo debe aplicarse cuando:

- Todos los hilos están esperando por la misma condición
- Sólo un hilo como mucho se puede beneficiar de que la condición se cumpla
- Esto es contractualmente cierto para todas las posibles subclases

Análisis del código: se notifica a los hilos que despierten

```
synchronized void
doWhenCondition()
{
    while (!condition)
        wait();
    //hace algo cuando
condición es verdadera
}
```

```
    // se termina de usar
    // el recurso compartido
    notifyall();
    …
}
```

Qué hace exactamente notifyAll()

1) **Identifica todos los hilos que están en espera (estado WAITING o TIMED_WAITING) sobre el mismo monitor del objeto.**

   (Es decir, hilos que previamente ejecutaron obj.wait() dentro de un bloque sincronizado sobre obj.)

2) **Los saca del conjunto de espera y los pasa al conjunto de "bloqueados por el monitor" (monitor entry set).**

   Es decir, los "despierta", pero no pueden continuar todavía, porque aún no tienen el monitor.

3) **El hilo que ejecutó notifyAll() mantiene el monitor hasta que sale del bloque synchronized o del método sincronizado.**

   Por tanto, los hilos despertados siguen bloqueados, esperando que el monitor se libere.

4) **Una vez que el hilo emisor libera el monitor, los hilos despertados compiten por adquirirlo.**

   Solo uno logrará entrar primero (el resto queda bloqueado de nuevo).

   Ese hilo retomará su ejecución exactamente después del wait().

5) **Importante: el hilo reanudado no reingresa al método desde el principio, sino que continúa después del wait() y vuelve a evaluar la condición del bucle while (!condicion).**

Análisis del código: se notifica a los hilos que despierten

```
synchronized void
doWhenCondition()
{
    while (!condition)
        wait();
    //hace algo cuando
condición es verdadera
}
```

```
    // se termina de usar
    // el recurso compartido
    notifyall();
    …
}
```

Qué pasa internamente

1) **T0 ejecuta notifyAll()**

   Todos los hilos que estaban en wait() sobre el mismo objeto obj pasan de estado: WAITING → BLOCKED (en la cola de entrada al monitor)

   Están despiertos, pero no pueden continuar todavía porque T0 aún tiene el monitor.

2) **T0 sale del bloque synchronized**

   Libera el monitor de obj.

3) **El sistema (la JVM) elige uno de los hilos despertados (por política interna del planificador) para que adquiera el monitor.**

4) **Ese hilo (digamos T1) entra al monitor**

   Retoma la ejecución justo después del wait().

   Vuelve a evaluar el while (!condicion):

   Si la condición ahora es false, sale del bucle y continúa.

   Si sigue siendo true, hace otro wait() y vuelve a dormirse.

5) **Cuando T1 sale del monitor, se libera de nuevo el lock y:**

   Otro hilo despertado (T2) puede entrar y hacer lo mismo.

   Luego otro (T3), etc.



Tread
consumidor



Único
Recurso
Compartido



Tread
productor



Tread
consumidor

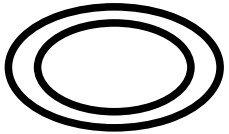


Tread
consumidor



Tread
consumidor





Único
Recurso
Compartido





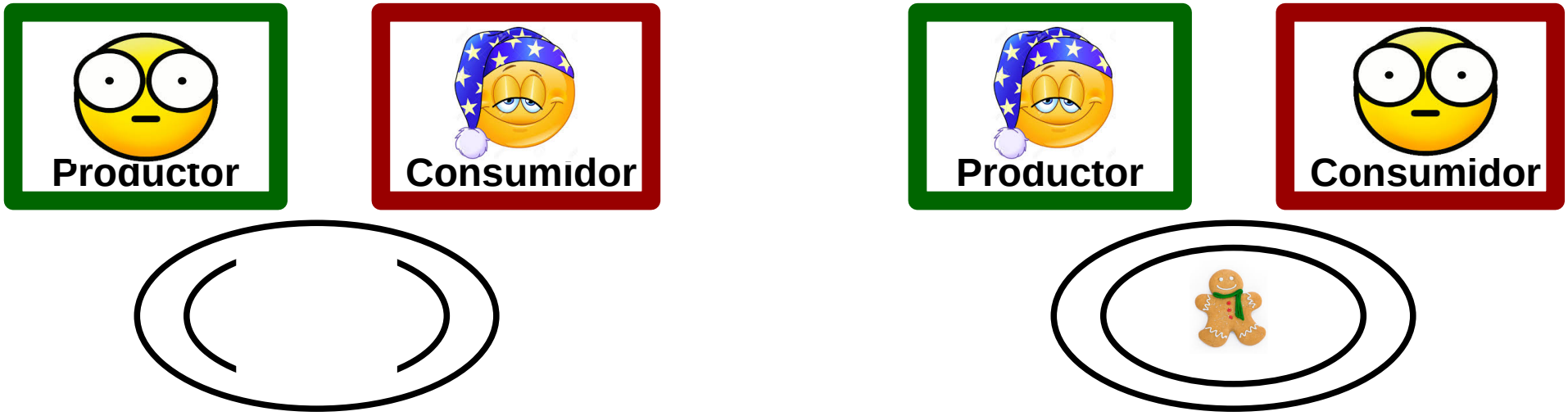





```
synchronized void doWhenCondition()
{
    while (!condition)
    wait();
//hace algo cuando la condición es verdadera
}
```



Ejemplo: Productor – Consumidor completo



El objetivo del ejemplo es generar un esquema de productor – consumidor, en donde cada hilo, compite por el uso del recurso compartido (CubbyHole).
En este caso, si el hilo no puede adquirir el recurso, en vez de terminar, espera hasta que esté disponible.
Para cada hilo, la condición para utilizar el recurso compartido es la opuesta.
Si el CubbyHole tiene un elemento, el hilo consumidor (Consumer) puede actuar y el productor (Producer) espera.
En el caso contrario, el hilo productor puede actuar y el consumidor espera.

Ejemplo: Productor – Consumidor completo

```
public class CubbyHole
{
    private int contents;
    private boolean available = false;

    public synchronized int get()
    {
        while (available == false)
        {
            try
            {
                wait();
            }
            catch (InterruptedException e)
            {
            }
        }
        available = false;
        notifyAll();
        return contents;
    }

    public synchronized void put(int value)
    {
        while (available == true)
        {
            try
            {
                wait();
            }
            catch (InterruptedException e)
            {
            }
        }
        contents = value;
        available = true;
        notifyAll();
    }
}
```

Ejemplo: Productor – Consumidor completo



Las otras clases (los hilos) no sufren cambios

La sentencia **syncronized**

No es lo mismo que el método sincronizado. Puede usarse para ámbitos menores que la invocación completa del método.

```
metodo(...)
{ …
  syncronized(ref a obj)
  {
     sentencias
  }
  …
}
```

Tiene dos partes:

- Un objeto cuyo bloqueo se va a adquirir
- Un grupo de sentencias que se ejecuta cuando se adquiere el bloqueo

La sentencia **syncronized**

```
syncronized(ref a obj)
{
   sentencias
}
```

Cuando se obtiene el bloqueo, se ejecutan las sentencias del bloque, y al final del bloque el bloqueo se libera.

Si se produce una excepción en el interior del bloque, el bloqueo también se libera.

NOTA: el *método synchronized* es simplemente una abreviatura de un método cuyo cuerpo está envuelto en una sentencia syncronized con una referencia *this*

La sentencia **syncronized**: ejemplo

Método (propio del Thread, No del recurso compartido) para sustituir los elementos de un array con sus valores absolutos, que se basa en la sentencia synchronized para controlar el acceso al arreglo.

```
public static void abs(int[] valores)
{
    synchronized(valores)
    {
        for(int i=0; i < valores.length; i++)
        {
            if(valores[i] < 0)
                valores[i] = - valores[i];
        }
    }
}
```

La sentencia **syncronized**: ejemplo

Esto es un ejemplo de lo que se conoce como sincronización de parte del cliente. Todos los clientes que utilizan el objeto compartido (arreglo en este caso) están de acuerdo en sincronizarse sobre ese objeto antes de manejarlo.

```
public static void abs(int[] valores)
{
    synchronized(valores)
    {
        for(int i=0; i < valores.length; i++)
        {
            if(valores[i] < 0)
                valores[i] = - valores[i];
        }
    }
}
```

La sentencia **syncronized**: ventajas y desventajas

La sentencia synchronized tiene ventajas sobre la sincronización de métodos:



- Definir una región de código sincronizada más pequeña que un método.
- Permite sincronizar sobre objetos diferentes de this.
- Bloqueo selectivo. Permite sincronizar entre sí, componentes de una clase. Permite una sincronización “de a grupos”.



PERO

Si se la usa como parte del código del cliente (Thread) se está delegando la responsabilidad del buen uso al Thread. El objeto no se puede “defender” dentro de un ambiente concurrente.



PERO PERO

Se puede usar la sentencia synchronized dentro de un recurso compartido también.

La sentencia **syncronized**: bloqueo selectivo

```
class GruposSeparados
{
    private double valA = 0.0;
    private double valB = 1.1;
    protected Object bloqueoA = new
    protected Object bloqueoB = new

    public double getA()
    {
        synchronized ( bloqueoA )
        {
            return valA;
        }
    }

    public void setA(double val)
    {
        synchronized ( bloqueoA )
        {
            valA = val;
        }
    }

    public double getB()
    {
        synchronized ( bloqueoB )
        {
            return valB;
        }
    }

    public void setB(double val)
    {
        synchronized ( bloqueoB )
        {
            valB = val;
        }
    }

    public double inicializar()
    {
        synchronized ( bloqueoA )
        {
            synchronized ( bloqueoB )
            {
                valA = valB = 0.0;
            }
        }
    }
}
```